

Legal Research System - Backend Documentation

Table of Contents

1. [Project Overview](#project-overview)
2. [Technology Stack](#technology-stack)
3. [Installation & Setup](#installation--setup)
4. [Project Structure](#project-structure)
5. [Database Schema](#database-schema)
6. [API Endpoints](#api-endpoints)
7. [Authentication](#authentication)
8. [File Processing Pipeline](#file-processing-pipeline)
9. [AI/ML Integration with Ollama](#aiml-integration-with-ollama)
10. [Error Handling](#error-handling)
11. [Performance Optimization](#performance-optimization)
12. [Deployment](#deployment)

Project Overview

The Legal Research System backend is a robust API service built with Node.js and Express.js that manages legal document storage, indexing, searching, and AI-powered analysis using Ollama for semantic understanding. It provides comprehensive endpoints for judgment retrieval, similarity analysis, and document processing.

Key Objectives

- Store and index legal judgments efficiently
- Provide fast full-text and semantic search capabilities
- Process and extract text from uploaded documents
- Perform similarity analysis using AI models
- Manage user data and authentication
- Generate embeddings for semantic search

Technology Stack

Backend Framework

- **Node.js 18+**: JavaScript runtime
- **Express.js 4.x**: Web framework
- **TypeScript** (Optional): Type safety

Database

- **PostgreSQL 14+**: Relational database

- **pgvector**: PostgreSQL extension for vector operations
- **Sequelize/TypeORM**: ORM layer

AI & ML

- **Ollama**: Local LLM for embeddings and analysis
- **node-ollama**: Node.js Ollama client

File Processing

- **multer**: File upload handling
- **pdf-parse**: PDF text extraction
- **sharp**: Image processing

Utilities

- **Joi**: Input validation
- **jsonwebtoken**: JWT authentication
- **dotenv**: Environment configuration
- **winston**: Logging
- **axios**: HTTP client

Development Tools

- **Nodemon**: Development server auto-reload
- **ESLint**: Code quality

- **Prettier**: Code formatting

Installation & Setup

Prerequisites

- Node.js 18.0.0 or higher
- PostgreSQL 14.0 or higher
- Ollama installed and running
- npm or yarn package manager

Step 1: Clone Repository

```
git clone <repository-url> cd legal-research-backend
```

Step 2: Install Dependencies

```
npm install # or yarn install
```

Step 3: Setup PostgreSQL Database

Install pgvector Extension

```
# Connect to PostgreSQL psql -U postgres # Create database CREATE DATABASE  
legal_research_db; # Connect to new database \c legal_research_db # Install  
pgvector extension CREATE EXTENSION IF NOT EXISTS vector; # Verify  
installation SELECT * FROM pg_extension;
```

Step 4: Environment Variables

Create a `.env` file in the root directory:

```
# Server Configuration NODE_ENV=development PORT=5000
API_BASE_URL=http://localhost:5000 # Database Configuration DB_HOST=localhost
DB_PORT=5432 DB_NAME=legal_research_db DB_USER=postgres
DB_PASSWORD=your_password DB_POOL_MIN=2 DB_POOL_MAX=10 # Ollama Configuration
OLLAMA_BASE_URL=http://localhost:11434 OLLAMA_MODEL=mistral
OLLAMA_EMBEDDING_MODEL=nomic-embed-text # JWT Configuration
JWT_SECRET=your_jwt_secret_key_here JWT_EXPIRY=24h REFRESH_TOKEN_EXPIRY=7d #
File Upload Configuration UPLOAD_DIR=./uploads MAX_FILE_SIZE=52428800
ALLOWED_FILE_TYPES=pdf,txt,doc,docx # Search Configuration
MAX_SEARCH_RESULTS=50 SEARCH_BATCH_SIZE=10 SIMILARITY_THRESHOLD=0.5 # Logging
LOG_LEVEL=debug LOG_DIR=./logs # CORS Configuration
CORS_ORIGIN=http://localhost:5173,http://localhost:3000 # Email Configuration
(Optional) SMTP_HOST=smtp.gmail.com SMTP_PORT=587
SMTP_USER=your_email@gmail.com SMTP_PASSWORD=your_app_password
```

Step 5: Database Migrations

```
npm run migrate:up
```

Step 6: Start Ollama Server

```
ollama serve # In another terminal, pull required model ollama pull mistral
ollama pull nomic-embed-text
```

Step 7: Run Development Server

```
npm run dev
```

The API will be available at `http://localhost:5000`

Project Structure

```
legal-research-backend/ ■■■■ src/ ■ ■■■■ config/ ■ ■ ■■■■ database.js ■ ■
■■■■ ollama.js ■ ■ ■■■■ environment.js ■ ■■■■ controllers/ ■ ■ ■■■■
searchController.js ■ ■ ■■■■ judgmentController.js ■ ■ ■■■■
uploadController.js ■ ■ ■■■■ userController.js ■ ■ ■■■■
comparisonController.js ■ ■■■■ models/ ■ ■ ■■■■ Judgment.js ■ ■ ■■■■ User.js
■ ■ ■■■■ SearchQuery.js ■ ■ ■■■■ Favorite.js ■ ■ ■■■■ JudgmentEmbedding.js ■
■■■■ services/ ■ ■ ■■■■ searchService.js ■ ■ ■■■■ judgmentService.js ■ ■
■■■■ ollamaService.js ■ ■ ■■■■ fileProcessingService.js ■ ■ ■■■■
embeddingService.js ■ ■ ■■■■ comparisonService.js ■ ■■■■ middleware/ ■ ■
■■■■ auth.js ■ ■ ■■■■ errorHandler.js ■ ■ ■■■■ validation.js ■ ■ ■■■■
uploadHandler.js ■ ■ ■■■■ rateLimiter.js ■ ■■■■ routes/ ■ ■ ■■■■
searchRoutes.js ■ ■ ■■■■ judgmentRoutes.js ■ ■ ■■■■ uploadRoutes.js ■ ■ ■■■■
userRoutes.js ■ ■ ■■■■ index.js ■ ■■■■ utils/ ■ ■ ■■■■ logger.js ■ ■ ■■■■
errorResponse.js ■ ■ ■■■■ validators.js ■ ■ ■■■■ constants.js ■ ■■■■ jobs/ ■
■■■■ processingQueue.js ■ ■ ■■■■ scheduledJobs.js ■ ■■■■ app.js ■■■■
migrations/ ■ ■■■■ 001_create_users_table.js ■ ■■■■
002_create_judgments_table.js ■ ■■■■ 003_create_embeddings_table.js ■ ■■■■
004_create_indexes.js ■■■■ tests/ ■ ■■■■ unit/ ■ ■■■■ integration/ ■ ■■■■
fixtures/ ■■■■ .env ■■■■ .env.example ■■■■ .eslintrc.js ■■■■ .prettierrc ■■■■
docker-compose.yml ■■■■ Dockerfile ■■■■ package.json ■■■■ README.md
```

Database Schema

Users Table

```
CREATE TABLE users ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), email
VARCHAR(255) UNIQUE NOT NULL, password_hash VARCHAR(255) NOT NULL, first_name
VARCHAR(100), last_name VARCHAR(100), role VARCHAR(50) DEFAULT 'user',
is_active BOOLEAN DEFAULT true, created_at TIMESTAMP DEFAULT
CURRENT_TIMESTAMP, updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP, CONSTRAINT
email_format CHECK (email LIKE '%@%.%') ); CREATE INDEX idx_users_email ON
users(email);
```

Judgments Table

```
CREATE TABLE judgments ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
case_number VARCHAR(100) UNIQUE NOT NULL, title VARCHAR(500) NOT NULL,
judge_name VARCHAR(255), court_name VARCHAR(255), bench VARCHAR(255),
judgment_date DATE, decision_date DATE, case_type VARCHAR(100), category
VARCHAR(100), content TEXT NOT NULL, extracted_text TEXT, summary TEXT,
legal_principles TEXT[], citations TEXT[], keywords TEXT[], file_path
VARCHAR(500), file_type VARCHAR(20), indexed_at TIMESTAMP, created_by UUID
REFERENCES users(id), created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP, searchable_text TSVECTOR,
metadata JSONB ); CREATE INDEX idx_judgments_case_number ON
judgments(case_number); CREATE INDEX idx_judgments_court ON
judgments(court_name); CREATE INDEX idx_judgments_date ON
judgments(judgment_date); CREATE INDEX idx_judgments_searchable ON judgments
USING GIN(searchable_text);
```

Embeddings Table (with pgvector)

```
CREATE TABLE judgment_embeddings ( id UUID PRIMARY KEY DEFAULT
gen_random_uuid(), judgment_id UUID NOT NULL REFERENCES judgments(id) ON
DELETE CASCADE, embedding_type VARCHAR(50), -- 'title', 'content', 'summary'
embedding vector(768), -- 768-dimensional embedding from Ollama created_at
TIMESTAMP DEFAULT CURRENT_TIMESTAMP, UNIQUE(judgment_id, embedding_type) );
CREATE INDEX idx_embeddings_judgment ON judgment_embeddings(judgment_id);
CREATE INDEX idx_embeddings_vector ON judgment_embeddings USING
ivfflat(embedding vector_cosine_ops);
```

Search Queries Table

```
CREATE TABLE search_queries ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
user_id UUID REFERENCES users(id), query TEXT NOT NULL, filters JSONB,
result_count INTEGER, execution_time_ms INTEGER, created_at TIMESTAMP DEFAULT
CURRENT_TIMESTAMP ); CREATE INDEX idx_search_user ON search_queries(user_id);
CREATE INDEX idx_search_date ON search_queries(created_at);
```

Favorites Table

```
CREATE TABLE favorites ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE, judgment_id
UUID NOT NULL REFERENCES judgments(id) ON DELETE CASCADE, created_at
TIMESTAMP DEFAULT CURRENT_TIMESTAMP, UNIQUE(user_id, judgment_id) ); CREATE
INDEX idx_favorites_user ON favorites(user_id);
```

API Endpoints

Authentication Endpoints

POST /api/auth/register

Register a new user

```
curl -X POST http://localhost:5000/api/auth/register \ -H "Content-Type:
application/json" \ -d '{ "email": "user@example.com", "password":
"securePassword123", "firstName": "John", "lastName": "Doe" }'
```

POST /api/auth/login

User login

```
curl -X POST http://localhost:5000/api/auth/login \ -H "Content-Type:
application/json" \ -d '{ "email": "user@example.com", "password":
"securePassword123" }'
```

Response:

```
{ "success": true, "data": { "token": "eyJhbGciOiJIUzI1NiIs...",
"refreshToken": "eyJhbGciOiJIUzI1NiIs...", "user": { "id": "uuid", "email":
"user@example.com", "firstName": "John", "lastName": "Doe" } } }
```


Search Endpoints

GET /api/search

Full-text search

```
curl -X GET 'http://localhost:5000/api/search?q=property+rights&court=High%20Court&limit=20&offset=0' \ -H "Authorization: Bearer token"
```

POST /api/search/advanced

Advanced search with multiple filters

```
curl -X POST http://localhost:5000/api/search/advanced \ -H "Content-Type: application/json" \ -H "Authorization: Bearer token" \ -d '{ "query": "property rights", "filters": { "court": "High Court", "year_from": 2015, "year_to": 2024, "case_type": "civil", "judge": "Justice Smith" }, "sort": "relevance", "limit": 20 }'
```

GET /api/search/suggestions

Get search suggestions

```
curl -X GET 'http://localhost:5000/api/search/suggestions?q=prop' \ -H "Authorization: Bearer token"
```

Judgment Endpoints

GET /api/judgments/:id

Get judgment details

```
curl -X GET http://localhost:5000/api/judgments/uuid \ -H "Authorization: Bearer token"
```

GET /api/judgments/:id/similar

Get similar judgments

```
curl -X GET 'http://localhost:5000/api/judgments/uuid/similar?limit=5' \ -H "Authorization: Bearer token"
```

GET /api/judgments/:id/citations

Get judgment citations

```
curl -X GET http://localhost:5000/api/judgments/uuid/citations \ -H "Authorization: Bearer token"
```

Upload Endpoints

POST /api/upload

Upload a judgment document

```
curl -X POST http://localhost:5000/api/upload \ -H "Authorization: Bearer token" \ -F "file=@judgment.pdf"
```

GET /api/upload/status/:uploadId

Check upload/processing status

```
curl -X GET http://localhost:5000/api/upload/status/uploadId \ -H "Authorization: Bearer token"
```

Comparison Endpoints

POST /api/comparisons

Compare multiple judgments

```
curl -X POST http://localhost:5000/api/comparisons \ -H "Content-Type: application/json" \ -H "Authorization: Bearer token" \ -d '{ "judgmentIds": ["uuid1", "uuid2", "uuid3"], "analysisType": "full" }'
```

Response:

```
{ "success": true, "data": { "comparison": { "similarities": [...], "differences": [...], "commonPrinciples": [...], "timeline": [...] } } }
```

Favorites Endpoints

POST /api/favorites/:judgmentId

Add judgment to favorites

```
curl -X POST http://localhost:5000/api/favorites/uuid \ -H "Authorization: Bearer token"
```

DELETE /api/favorites/:judgmentId

Remove from favorites

```
curl -X DELETE http://localhost:5000/api/favorites/uuid \ -H "Authorization: Bearer token"
```

GET /api/favorites

Get user's favorite judgments

```
curl -X GET http://localhost:5000/api/favorites \ -H "Authorization: Bearer token"
```

Authentication

JWT Implementation

****File**:** `src/middleware/auth.js`

```
const jwt = require('jsonwebtoken'); const { UnauthorizedError } =
require('../utils/errorResponse'); module.exports.verifyToken = (req, res,
next) => { try { const token = req.headers.authorization?.split(' ')[1]; if
(!token) { throw new UnauthorizedError('No token provided'); } const decoded
= jwt.verify(token, process.env.JWT_SECRET); req.userId = decoded.userId;
req.user = decoded; next(); } catch (error) { if (error instanceof
jwt.TokenExpiredError) { return res.status(401).json({ success: false, error:
'Token expired' }); } next(error); } }; module.exports.generateTokens =
(userId) => { const token = jwt.sign( { userId, type: 'access' },
process.env.JWT_SECRET, { expiresIn: process.env.JWT_EXPIRY } ); const
refreshToken = jwt.sign( { userId, type: 'refresh' }, process.env.JWT_SECRET,
{ expiresIn: process.env.REFRESH_TOKEN_EXPIRY } ); return { token,
refreshToken }; };
```

File Processing Pipeline

Document Upload and Processing

****File**:** `src/services/fileProcessingService.js`

```
const fs = require('fs').promises; const path = require('path'); const
pdfParse = require('pdf-parse'); const { logger } =
require('../utils/logger'); class FileProcessingService { async
processJudgmentFile(filePath, fileType) { try { let extractedText; if
(fileType === 'pdf') { extractedText = await this.extractPdfText(filePath); }
else if (fileType === 'txt') { extractedText = await fs.readFile(filePath,
'utf-8'); } else { throw new Error(`Unsupported file type: ${fileType}`); }
// Clean and normalize text const cleanedText =
this.cleanText(extractedText); // Extract metadata const metadata = await
```

```

this.extractMetadata(cleanedText); // Extract legal principles and keywords
const principles = await this.extractLegalPrinciples(cleanedText); const
keywords = this.extractKeywords(cleanedText); return { extractedText:
cleanedText, metadata, principles, keywords, textLength: cleanedText.length,
processingTime: Date.now() }; } catch (error) { logger.error(`File processing
error: ${error.message}`); throw error; } } async extractPdfText(filePath) {
const dataBuffer = await fs.readFile(filePath); const data = await
pdfParse(dataBuffer); return data.text; } cleanText(text) { return text
.replace(/\s+/g, ' ') .replace(/[^\w\s\d\-\.,;:()]/g, '') .trim(); } async
extractMetadata(text) { // Extract key metadata using regex patterns const
patterns = { caseNumber: /Case\s*(?:No|Number)[:\s]+([A-Z0-9\-\./]+)/i, judge:
/(?:Justice|Hon'ble|Honourable)\s+([A-Z][a-z]+(?:\s+[A-Z][a-z]+)*)/g, court:
/(?:Court|Bench):\s*([^\n]+)/i, date:
/Date[:\s]+(\d{1,2}[\-\/]\d{1,2}[\-\/]\d{4})/ }; const metadata = {}; for
(const [key, pattern] of Object.entries(patterns)) { const match =
text.match(pattern); if (match) { metadata[key] = match[1]; } } return
metadata; } extractKeywords(text) { // Simple keyword extraction - can be
enhanced with NLP const words = text.split(/\s+/) .filter(word => word.length
> 4) .map(word => word.toLowerCase()); const frequency = {};
words.forEach(word => { frequency[word] = (frequency[word] || 0) + 1; });
return Object.entries(frequency) .sort((a, b) => b[1] - a[1]) .slice(0, 20)
.map(([word]) => word); } async extractLegalPrinciples(text) { // Use Ollama
to extract legal principles const ollamaService = require('./ollamaService');
const prompt = `Extract key legal principles from this
judgment:\n\n${text.substring(0, 1000)}`; const principles = await
ollamaService.analyze(prompt); return principles; } } module.exports = new
FileProcessingService();

```

AI/ML Integration with Ollama

Ollama Service

****File**:** `src/services/ollamaService.js`

```

const axios = require('axios'); const { logger } =
require('../utils/logger'); class OllamaService { constructor() {
this.baseUrl = process.env.OLLAMA_BASE_URL || 'http://localhost:11434';
this.model = process.env.OLLAMA_MODEL || 'mistral'; this.embeddingModel =
process.env.OLLAMA_EMBEDDING_MODEL || 'nomic-embed-text'; } async
generateEmbedding(text) { try { const response = await
axios.post(`${this.baseUrl}/api/embeddings`, { model: this.embeddingModel,

```

```

prompt: text }); return response.data.embedding; } catch (error) {
  logger.error(`Embedding generation failed: ${error.message}`); throw new
  Error('Failed to generate embedding'); } } async generateAnalysis(text,
  analysisType = 'summary') { try { let prompt; if (analysisType === 'summary')
  { prompt = `Provide a concise summary of this legal judgment:\n\n${text}`; }
  else if (analysisType === 'principles') { prompt = `Extract the key legal
  principles from this judgment:\n\n${text}`; } else if (analysisType ===
  'comparison') { prompt = `Analyze the similarities and differences in legal
  reasoning:\n\n${text}`; } const response = await
  axios.post(`${this.baseUrl}/api/generate`, { model: this.model, prompt:
  prompt, stream: false }); return response.data.response; } catch (error) {
  logger.error(`Analysis generation failed: ${error.message}`); throw error; }
  } async findSimilarJudgments(embedding, limit = 5, threshold = 0.5) { try {
  const { Judgment, JudgmentEmbedding } = require('../models'); const results =
  await JudgmentEmbedding.sequelize.query(` SELECT j.id, j.title,
  j.case_number, 1 - (e.embedding <=> $1::vector) as similarity_score FROM
  judgment_embeddings e JOIN judgments j ON e.judgment_id = j.id WHERE 1 -
  (e.embedding <=> $1::vector) > $2 ORDER BY similarity_score DESC LIMIT $3 `,
  { bind: [JSON.stringify(embedding), threshold, limit], type: 'SELECT' });
  return results; } catch (error) { logger.error(`Similar judgment search
  failed: ${error.message}`); throw error; } } async
  batchGenerateEmbeddings(texts) { try { const embeddings = await Promise.all(
  texts.map(text => this.generateEmbedding(text)) ); return embeddings; } catch
  (error) { logger.error(`Batch embedding generation failed:
  ${error.message}`); throw error; } } } module.exports = new OllamaService();

```

Embedding Service

****File**:** `src/services/embeddingService.js`

```

const { JudgmentEmbedding, Judgment } = require('../models'); const
ollamaService = require('../ollamaService'); const { logger } =
require('../utils/logger'); class EmbeddingService { async
generateAndStoreEmbeddings(judgmentId) { try { const judgment = await
Judgment.findPk(judgmentId); if (!judgment) { throw new Error(`Judgment
${judgmentId} not found`); } // Generate embeddings for title, content, and
summary const embeddings = await Promise.all([
ollamaService.generateEmbedding(judgment.title),
ollamaService.generateEmbedding(judgment.extracted_text.substring(0, 2000)),
judgment.summary ? ollamaService.generateEmbedding(judgment.summary) : null
]); // Store embeddings const [titleEmbedding, contentEmbedding,
summaryEmbedding] = embeddings; await JudgmentEmbedding.create({ judgmentId,
embeddingType: 'title', embedding: titleEmbedding }); await
JudgmentEmbedding.create({ judgmentId, embeddingType: 'content', embedding:
contentEmbedding }); if (summaryEmbedding) { await JudgmentEmbedding.create({
judgmentId, embeddingType: 'summary', embedding: summaryEmbedding }); }
logger.info(`Embeddings generated for judgment ${judgmentId}`); return true;
} catch (error) { logger.error(`Embedding generation error:

```

```

    ${error.message}`); throw error; } } async
    findSimilarBySemanticSimilarity(judgmentId, limit = 5) { try { const
    contentEmbedding = await JudgmentEmbedding.findOne({ where: { judgmentId,
    embeddingType: 'content' } }); if (!contentEmbedding) { throw new
    Error('Embedding not found'); } const similar = await
    ollamaService.findSimilarJudgments( contentEmbedding.embedding, limit );
    return similar; } catch (error) { logger.error(`Semantic similarity search
    failed: ${error.message}`); throw error; } } } module.exports = new
    EmbeddingService();

```

Error Handling

Custom Error Classes

****File**:** `src/utils/errorResponse.js`

```

class ApiError extends Error { constructor(message, statusCode = 500) {
super(message); this.statusCode = statusCode; this.isOperational = true; } }
class ValidationError extends ApiError { constructor(message) {
super(message, 400); } } class UnauthorizedError extends ApiError {
constructor(message = 'Unauthorized') { super(message, 401); } } class
ForbiddenError extends ApiError { constructor(message = 'Forbidden') {
super(message, 403); } } class NotFoundError extends ApiError {
constructor(message = 'Resource not found') { super(message, 404); } }
module.exports = { ApiError, ValidationError, UnauthorizedError,
ForbiddenError, NotFoundError };

```

Global Error Handler Middleware

****File**:** `src/middleware/errorHandler.js`

```

const { logger } = require('../utils/logger'); module.exports = (err, req,
res, next) => { logger.error(err); const statusCode = err.statusCode || 500;
const message = err.message || 'Internal Server Error';
res.status(statusCode).json({ success: false, error: { message, statusCode,
...(process.env.NODE_ENV === 'development' && { stack: err.stack }) } }); };

```

Performance Optimization

Database Query Optimization

```
// Use indexes on frequently queried columns CREATE INDEX
idx_judgments_full_text ON judgments USING GIN(searchable_text); CREATE INDEX
idx_judgments_date ON judgments(judgment_date DESC); CREATE INDEX
idx_embeddings_similarity ON judgment_embeddings USING ivfflat(embedding
vector_cosine_ops); // Use connection pooling const pool = new Pool({ max:
20, idleTimeoutMillis: 30000, connectionTimeoutMillis: 2000, });
```

Caching Strategy

```
const redis = require('redis'); const client = redis.createClient(); // Cache
search results for 1 hour const searchCache = async (query, filters) => {
const key = `search:${query}:${JSON.stringify(filters)}`; const cached =
await client.get(key); if (cached) return JSON.parse(cached); const results =
await performSearch(query, filters); await client.setex(key, 3600,
JSON.stringify(results)); return results; };
```

Rate Limiting

```
const rateLimit = require('express-rate-limit'); const searchLimiter =
rateLimit({ windowMs: 15 * 60 * 1000, // 15 minutes max: 100, // limit each
IP to 100 requests per windowMs message: 'Too many search requests, please
try again later' }); app.use('/api/search', searchLimiter);
```

Deployment

Docker Setup

****Dockerfile****:

```
FROM node:18-alpine WORKDIR /app # Install dependencies COPY package*.json ./
RUN npm ci --only=production # Copy application COPY src ./src COPY
migrations ./migrations # Expose port EXPOSE 5000 # Health check HEALTHCHECK
--interval=30s --timeout=3s --start-period=5s --retries=3 \ CMD node -e
"require('http').get('http://localhost:5000/health', (r) => r.statusCode ===
200 ? process.exit(0) : process.exit(1))" # Start application CMD ["node",
"src/app.js"]
```

****docker-compose.yml****:

```
version: '3.8' services: postgres: image: postgres:14-alpine environment:
POSTGRES_DB: legal_research_db POSTGRES_USER: postgres POSTGRES_PASSWORD:
postgres volumes: - postgres_data:/var/lib/postgresql/data ports: -
"5432:5432" ollama: image: ollama/ollama ports: - "11434:11434" volumes: -
ollama_data:/root/.ollama backend: build: . environment: DB_HOST: postgres
DB_USER: postgres DB_PASSWORD: postgres OLLAMA_BASE_URL: http://ollama:11434
ports: - "5000:5000" depends_on: - postgres - ollama volumes: -
./uploads:/app/uploads volumes: postgres_data: ollama_data:
```

Running with Docker

```
docker-compose up -d
```

Environment-Specific Configuration

Production `.env`:

```
NODE_ENV=production DB_HOST=prod-db.example.com DB_SSL_MODE=require
LOG_LEVEL=info JWT_SECRET=<secure-random-key>
```

Best Practices

1. **Security**: Always validate and sanitize user input
2. **Performance**: Use database indexes on frequently queried columns
3. **Logging**: Log all errors and important operations
4. **Testing**: Implement comprehensive unit and integration tests
5. **Documentation**: Keep API documentation updated
6. **Monitoring**: Set up error tracking and performance monitoring
7. **Backup**: Regular database backups
8. **Versioning**: Use API versioning (/api/v1/...)

Monitoring & Maintenance

Health Check Endpoint

```
app.get('/health', (req, res) => { res.json({ status: 'ok', timestamp: new Date(), uptime: process.uptime() }); });
```

Performance Monitoring

```
const promClient = require('prom-client'); const httpRequestDuration = new promClient.Histogram({ name: 'http_request_duration_seconds', help: 'Duration of HTTP requests in seconds', labelNames: ['method', 'route', 'status_code'] }); app.use((req, res, next) => { const start = Date.now(); res.on('finish',
```

```
() => { const duration = (Date.now() - start) / 1000; httpRequestDuration
.labels(req.method, req.route?.path, res.statusCode) .observe(duration); });
next(); };
```

Support & Resources

- **Express.js Documentation**: expressjs.com
- **PostgreSQL Documentation**: [postgresql.org](https://www.postgresql.org/docs)
- **Ollama Documentation**: ollama.ai
- **Sequelize ORM**: sequelize.org
- **pgvector**: github.com/pgvector/pgvector

Last Updated: August 2024

Version: 1.0.0

Maintainer: Your Team Name